

Ramrao Adik Institute of Technology
Department of Computer Science and Engineering

TE LAB PROJECT REPORT

CivicLens AI: NLP System for Citizen Complaint Triage, Prioritization and Routing

Submitted by

Sanchit Patil (24AD1091)
Shinoy Negi (24AD1126)
Shravan Shewade (24AD1128)

Batch: B2

Guided by
Dr. Ekta Sarda

ABSTRACT

CivicLens AI is an NLP-based complaint triage system developed to demonstrate how machine learning and transparent rule-based analysis can automate the handling of citizen complaints in a municipal grievance system. From the raw complaint text alone, the system classifies the complaint into one of ten civic categories, estimates its urgency as Low, Medium, High or Critical, checks whether similar complaints have already been reported, and routes it to the responsible department with a service-level deadline.

The project is implemented as a Python package with a FastAPI REST service, SQLite persistence and a React frontend that contains a citizen complaint form and a staff triage dashboard. The category classifier uses TF-IDF features with a calibrated Linear SVM, and the urgency classifier combines TF-IDF with 11 engineered lexicon and text-style features and Logistic Regression. Both were trained on a synthetic dataset of 3,000 complaints with deliberately injected label noise and typing errors. On a held-out test set of 600 complaints, the category classifier reports 98.33% accuracy and the urgency classifier reports 94.50% accuracy with an ordinal mean absolute error of 0.055.

The project demonstrates how a lightweight and explainable pipeline, in which every priority score can be traced to the keywords, model confidences and repeat reports that produced it, can be built without depending on an external AI API for its core NLP features.

Project aspect	Summary
Domain	Natural Language Processing / Civic Technology
Application	FastAPI REST service with a React staff dashboard
Dataset	Synthetic complaint dataset (3,000 complaints, 10 categories)
Main ML models	TF-IDF + Linear SVM (category), TF-IDF + Logistic Regression (urgency)
Supporting NLP	Lexicon-based urgency scoring and TF-IDF duplicate detection
Routing logic	Department routing, SLA deadlines and critical escalation

TABLE OF CONTENTS

1. Introduction 4

2. Literature Survey 5

3. Limitations of Existing Systems 6

4. Problem Statement and Objectives 7

5. Dataset and Data Preparation 8

6. Project Flow and System Architecture 9

7. Technologies and Methodologies 10

8. System Design and Implementation 11

9. Application Features and User Workflow 12

10. Result and Analysis 13

11. Testing and Validation 15

12. Security and Privacy Considerations 16

13. Limitations and Future Scope 17

14. Conclusion 18

15. References 19

CHAPTER 1

INTRODUCTION

Municipal grievance systems receive a continuous stream of citizen complaints about potholes, water outages, garbage collection, street lighting and public safety. Many of them rely on citizens choosing a category from a drop-down list and on a human clerk deciding which complaints are urgent. In practice, citizens often pick the wrong category, urgent problems wait in the same queue as trivial ones, and the same problem is reported many times as separate, unrelated tickets.

CivicLens AI was developed as an NLP-based triage system that addresses these problems from the raw complaint text alone. The system tokenizes and stems the text, predicts the civic category using TF-IDF features and a Linear SVM, and estimates urgency with a second classifier that also uses features from a transparent keyword lexicon. A duplicate detector compares the complaint with earlier reports, and a routing engine combines all signals into a priority score, a responsible department and a service-level deadline.

The project goes beyond plain text classification by making every decision explainable. Each routed ticket carries an explanation that lists the keywords, model confidences and repeat reports behind its priority score, so that a reviewer can audit the outcome instead of trusting a black-box model.

1.1 Project scope

- Automatic classification of complaints into ten civic categories.
- Four-level urgency classification: Low, Medium, High and Critical.
- Transparent lexicon-based scoring of urgency and frustration.
- Repeat-complaint detection using TF-IDF cosine similarity.
- Department routing, SLA deadlines and escalation of critical complaints.
- FastAPI REST service with SQLite persistence.
- React interface with a citizen complaint form and a staff triage dashboard.

1.2 Significance

The project provides an academic implementation in which the major components can be inspected and tested independently. It demonstrates text preprocessing, feature extraction, supervised learning, rule-based scoring, model evaluation and full-stack integration in a single civic-technology application.

CHAPTER 2

LITERATURE SURVEY

The project draws from established work in text categorization, term weighting, sentiment lexicons and information retrieval. These areas provide the theoretical basis for the modules implemented in CivicLens AI.

Work	Contribution relevant to the project
Salton & Buckley (1988)	TF-IDF term weighting used to represent text as numerical features.
Porter (1980)	Suffix-stripping algorithm used to reduce complaint words to their stems.
McCallum & Nigam (1998)	Comparison of event models for Naive Bayes text classification.
Joachims (1998)	Support Vector Machines for high-dimensional text categorization.
Sebastiani (2002)	Survey of machine-learning methods for automated text categorization.
Hutto & Gilbert (2014)	VADER: a transparent, lexicon-based approach to sentiment analysis.
Manning et al. (2008)	Vector space model and cosine similarity for matching documents.
Gutiérrez et al. (2016)	Ordinal regression methods and evaluation measures for ordered classes.

2.1 Relation to CivicLens AI

The implementation uses TF-IDF as the common text representation for category classification, urgency classification and duplicate detection. Multinomial Naive Bayes, Logistic Regression and a Linear SVM are compared for category classification, following the text-categorization literature. Urgency and frustration scoring follows the lexicon-based approach of VADER, using a smaller domain-specific word list whose every match can be inspected. Because urgency levels are ordered, the evaluation also reports an ordinal mean absolute error in addition to accuracy.

CHAPTER 3

LIMITATIONS OF EXISTING SYSTEMS

Several practical limitations motivate the combined design of CivicLens AI. These limitations are considered from the perspective of the functionality targeted by the project.

- Manual category selection from a drop-down list often sends complaints to the wrong department.
- Urgency is judged by a human clerk, so dangerous complaints may wait in the same queue as trivial requests.
- The same problem reported by several citizens becomes several unrelated tickets, and repeat reports are not used as evidence of importance.
- Black-box classifiers do not explain why a complaint received a given priority, which is undesirable for a civic system that must be auditable.
- Keyword-only rules are brittle against typing mistakes, merged words and varied phrasing.
- Cloud-based NLP services need internet connectivity, API credentials and external processing of citizen data.

CivicLens AI responds by combining supervised classification, a transparent lexicon, duplicate detection and deterministic routing in a single local and explainable pipeline.

CHAPTER 4

PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: To build an NLP-based system that reads a free-text citizen complaint, identifies its civic category and urgency, detects repeat reports, and routes it to the responsible department with a service-level deadline while keeping every decision explainable.

4.1 Objectives

- Generate a reproducible, labelled complaint dataset with realistic label noise and typing errors.
- Preprocess complaint text using tokenization, stop-word removal and stemming.
- Compare Multinomial Naive Bayes, Logistic Regression and Linear SVM for category classification.
- Compare Logistic Regression, Linear SVM and Random Forest for urgency classification.
- Combine TF-IDF features with engineered lexicon features for urgency prediction.
- Detect repeat complaints using TF-IDF cosine similarity.
- Blend classifier, lexicon and repeat-report signals into a 0-100 priority score.
- Route tickets to departments with SLA deadlines and escalate Critical complaints.
- Provide a REST API and a web interface for citizens and civic staff.

4.2 Expected outcome

The expected outcome is a working demonstration of an NLP-based complaint triage system in which classification, prioritization, duplicate detection and routing are available through one API and one web application.

CHAPTER 5

DATASET AND DATA PREPARATION

The project uses a synthetic dataset generated by **civiclens/data/generate_dataset.py** and stored as **data/raw/complaints_dataset.csv**. Each record contains a complaint ID, the complaint text, its category and urgency labels, the ward, the street and a submission date. No real municipal dataset was available for redistribution, so complaints are composed from hand-written templates that are tiered by urgency, combined with wards, streets, durations, salutations and sign-offs using a fixed random seed.

Dataset property	Verified value
Total complaints	3,000 (all texts unique)
Categories	10 (289 to 308 complaints per category)
Urgency: Low / Medium / High / Critical	813 / 1,016 / 790 / 381 (27.1% / 33.9% / 26.3% / 12.7%)
Average complaint length	33.8 words (minimum 9, maximum 74)
Training / test split	2,400 / 600 (80% / 20%), stratified by category
Random state	42
Injected label noise	Urgency shifted by one level in 8%; category swapped within related pairs in 3%
Injected text noise	One or two dropped characters or merged words in about half of the complaints
TF-IDF vocabulary	3,363 unigram and bigram terms (category model)

5.1 Text preprocessing

Each complaint is lowercased and split into alphabetic tokens with a regular expression, which discards digits and punctuation. English stop-words from scikit-learn's bundled list and single-letter tokens are removed, and the remaining tokens are reduced to stems with the Porter stemmer. No NLTK corpora need to be downloaded, so the pipeline runs fully offline. The tokens are then converted to TF-IDF features using unigrams and bigrams, a minimum document frequency of two and sublinear term frequency.

5.2 Dataset observation

Every complaint text is unique, but the texts are assembled from a finite set of templates. As a result, 93.2% of the test complaints have a training complaint with a TF-IDF cosine similarity of at least 0.55, and 49.5% have one of at least 0.70. Label noise and typing errors were injected on purpose so that the metrics do not reach a trivial 100%.

Important: The dataset is synthetic. The reported metrics show how the pipeline behaves on template-based complaints and should not be interpreted as accuracy on real citizen-submitted text.

CHAPTER 6

PROJECT FLOW AND SYSTEM ARCHITECTURE

Stage	Operation	Output
1	Citizen submits the complaint text and an optional location	Raw complaint
2	Tokenization, stop-word removal and Porter stemming	Stemmed tokens
3	TF-IDF vectorization (unigrams and bigrams)	Sparse feature vector
4	Category classification (calibrated Linear SVM)	Category + confidence
5	Urgency classification (TF-IDF + 11 engineered features)	Urgency class + confidence
6	Lexicon analysis of urgency and frustration terms	Lexicon score + matched terms
7	Duplicate search inside the predicted category	Similar complaint IDs
8	Routing: priority blend, department, SLA and escalation	Routed ticket
9	Persistence in SQLite and delivery through the API	Stored ticket / dashboard view

6.1 Architectural layers

Presentation layer: React single-page application with a citizen complaint form and a staff triage dashboard.

Application layer: FastAPI routes, Pydantic request validation, CORS configuration and a lock around the shared analyzer.

ML/NLP layer: Preprocessing, TF-IDF classifiers, lexicon scoring, duplicate index and routing engine, orchestrated by the ComplaintAnalyzer class.

Data layer: SQLite database accessed through SQLAlchemy, storing every routed ticket together with its explanation.

6.2 Processing principle

The design combines three independent signals instead of trusting the classifier alone: the predicted urgency class, the transparent lexicon score and the number of similar complaints already reported. The priority score is 0.7 × classifier score + 0.3 × lexicon score, where the classifier score is 25, 50, 75 or 100 for Low, Medium, High and Critical and the lexicon score is scaled to 0-100. If two or more similar complaints exist, 12 points are added (capped at 100). Scores of at least 37.5, 62.5 and 87.5 map to Medium, High and Critical, with SLA deadlines of 72, 24 and 4 hours (168 hours for Low). Critical tickets are also escalated to the Disaster and Emergency Response Cell.

CHAPTER 7

TECHNOLOGIES AND METHODOLOGIES

Component	Technology / methodology
Programming language	Python 3 (backend and machine learning), JavaScript (frontend)
Machine learning	scikit-learn [10]
Feature extraction	TF-IDF with unigrams, bigrams and sublinear term frequency
Category classification	Multinomial Naive Bayes, Logistic Regression, Linear SVM
Urgency classification	Logistic Regression, Linear SVM, Random Forest [9]
NLP preprocessing	Regex tokenizer, scikit-learn stop-words, NLTK Porter stemmer
Rule-based analysis	Hand-authored urgency, dampener, frustration and calm lexicons
Duplicate detection	TF-IDF cosine similarity within the predicted category
Web framework	FastAPI, Uvicorn and Pydantic
Database	SQLite with SQLAlchemy 2.0
Frontend	React 19 and Vite

7.1 Category classification methodology

Complaint text is converted to TF-IDF features. Multinomial Naive Bayes, Logistic Regression and a Linear SVM are compared with 5-fold stratified cross-validation on macro-F1, and the best model is kept. The Linear SVM won; as it has no native probability output, it is wrapped in a calibrated classifier (sigmoid calibration [11], 3 folds) so that every prediction carries a confidence score.

7.2 Urgency classification methodology

The urgency classifier joins TF-IDF features with 11 engineered features: nine from the lexicon analysis (urgency and sentiment scores, counts of urgent, dampener and frustration terms, repeat flag, duration mentions, exclamation and capitalized-word counts) and two text-length measures. They are standardized and combined with the TF-IDF matrix in a FeatureUnion. Logistic Regression, Linear SVM and a 300-tree Random Forest were compared by cross-validation, and Logistic Regression was selected.

7.3 Lexicon and duplicate-detection methodology

The lexicon holds 37 critical terms, 22 elevated terms, 16 low-urgency dampeners, 20 frustration terms and 8 calm terms with small hand-set weights. Matches are summed into a raw urgency score and squashed to the range 0-1 with a logistic function. The duplicate detector keeps a TF-IDF matrix of all earlier complaints (3,000 at start-up) and returns up to five complaints of the same predicted category whose cosine similarity is at least 0.55; every new complaint is then added to the index.

CHAPTER 8

SYSTEM DESIGN AND IMPLEMENTATION

Source module	Role in the system
config.py	Paths, ten categories, urgency levels, SLA hours and department routing table.
generate_dataset.py	Synthetic dataset generator with label noise and typing-error injection.
preprocessing.py	Tokenization, stop-word removal and Porter stemming.
lexicons.py / features.py	Rule-based urgency and frustration scoring; 11 engineered features.
category_model.py / urgency_model.py	Category and urgency classifiers: model selection, training, prediction.
duplicates.py	TF-IDF similarity index for repeat-complaint detection.
router.py	Priority blending, department routing, SLA deadline and escalation.
pipeline.py	ComplaintAnalyzer: the path from raw text to a routed ticket.
main.py (api)	FastAPI endpoints, validation, persistence and dashboard statistics.

8.1 Database design (complaints table)

Field(s)	Purpose
complaint_id, text, location	Primary key, complaint text and optional location.
category, category_confidence	Predicted category and its confidence.
ml_predicted_urgency, lexicon_urgency_score	Classifier's urgency class and normalized lexicon score.
priority_score, final_urgency_level	Blended 0-100 score and the resulting urgency level.
department, escalated_department	Responsible department and, for Critical tickets, the emergency cell.
sla_hours, sla_deadline	Promised response time and its deadline.
duplicate_count, similar_complaint_ids, explanation	Repeat-report evidence and the audit explanation.
status, submitted_at	Workflow status (Open, In Progress, Resolved) and timestamp.

8.2 Persistence and concurrency

The application uses SQLAlchemy with SQLite by default; moving to PostgreSQL or MySQL only requires changing the database URL. The trained analyzer is loaded once at start-up, and a lock protects its in-memory duplicate index because that index is updated on every submission.

CHAPTER 9

APPLICATION FEATURES AND USER WORKFLOW

9.1 Complaint submission

On the Report an issue tab, a citizen describes the problem in their own words (10 to 4,000 characters) and may enter a location. The system returns a ticket showing the ticket ID, urgency level, priority score out of 100, category with confidence, routed department, SLA deadline, number of similar reports and the keywords that triggered the urgency flag.

9.2 Staff triage dashboard

The Staff dashboard tab shows summary tiles (total, open, in progress, resolved and average priority), breakdowns by category, urgency and department, and a complaints table sorted by priority score. Staff can filter by status, urgency, category and department.

9.3 Status workflow

Every ticket starts as Open. Staff change it to In Progress or Resolved from the dashboard, which calls the API and updates the stored record.

9.4 REST API

Method and path	Description
GET /health	Health check; reports whether the trained models are loaded.
POST /complaints	Submit a complaint and receive the routed ticket.
GET /complaints	List complaints with category, urgency, department and status filters.
GET /complaints/{id}	Fetch a single complaint.
PATCH /complaints/{id}/status	Set the status to Open, In Progress or Resolved.
GET /stats/dashboard	Aggregate counts for the staff dashboard.

9.5 Command-line demo and explainability

The script scripts/demo_cli.py analyzes complaints without the API or database. Every ticket carries an explanation listing the matched urgent, dampener and frustration terms, the classifier confidences, the blended score and any repeat-report boost.

CHAPTER 10

RESULT AND ANALYSIS

The models were evaluated on a held-out test set of 600 complaints (a 20% stratified split) that was not used for training or model selection. Model selection used 5-fold stratified cross-validation on the 2,400 training complaints only. The test-set results of the two selected models are shown below.

Task	Selected model	Accuracy	Macro Precision	Macro Recall	Macro F1
Category (10 classes)	Linear SVM (calibrated)	98.33%	0.984	0.983	0.983
Urgency (4 classes)	Logistic Regression	94.50%	0.951	0.939	0.944

10.1 Model selection

Task	Candidate model	Mean macro-F1 (5-fold CV)	Std
Category	Multinomial Naive Bayes	0.9796	0.0059
Category	Logistic Regression	0.9804	0.0054
Category	Linear SVM (selected)	0.9808	0.0055
Urgency	Logistic Regression (selected)	0.9377	0.0127
Urgency	Linear SVM	0.9371	0.0114
Urgency	Random Forest	0.9367	0.0123

The differences between candidates are smaller than the fold-to-fold standard deviation, so the candidate models perform practically equally on this dataset; the selected model is the one with the highest mean.

10.2 Interpretation

The category classifier misclassified 10 of 600 test complaints. Six categories were classified perfectly, and every error occurred inside two related pairs: Roads & Infrastructure with Street Lighting, and Sanitation & Waste Management with Health & Sanitation. These are the pairs that the dataset generator swaps when it injects category label noise.

The urgency classifier misclassified 33 of 600 complaints. Its ordinal mean absolute error is 0.055 and every error was to an adjacent level, so no Critical complaint was predicted as Low or Medium. Critical complaints reached a precision of 0.970 and a recall of 0.914: 64 of 70 were found and the other six were classified as High.

10.3 Independent reproduction

As an additional check, the dataset was re-read, the 80/20 split with random state 42 was regenerated (identical to the stored split) and both selected models were retrained from scratch with scikit-learn 1.9.1 without saving any files. This reproduced 98.33% category accuracy and 94.50% urgency accuracy with an ordinal MAE of 0.055, matching the committed evaluation report.

The complete routing decision was also evaluated. After blending the urgency classifier with the lexicon score (without the repeat-report boost), the final urgency level matched the label for 93.83% of the test complaints, slightly below the classifier alone. The blend changed 4 of 600 predictions (two Critical to High and two Low to Medium).

Evaluation caution: the dataset is synthetic and template-based, and 93.2% of the test complaints have a training complaint with a cosine similarity of at least 0.55, so accuracy on real citizen complaints is likely to be lower. The TF-IDF vectorizers are fitted inside the training pipeline, so no test data leaks into the models; only the duplicate index is deliberately built from the full corpus.

10.4 Sample pipeline outputs

Complaint (summary)	Category (confidence)	Level (score)	Routed to	SLA
Massive pothole, two-wheeler already crashed, "fix immediately"	Roads & Infrastructure (0.91)	Critical (98.6)	Public Works Department (PWD); escalated to Disaster & Emergency Response Cell	4 h
Overgrown park grass, "no rush"	Parks & Public Property (0.96)	Low (36.2)	Parks & Horticulture Department	168 h
No water for two days, elderly residents struggling, "emergency"	Water Supply (0.95)	Critical (97.1)	Municipal Water Board; escalated to Disaster & Emergency Response Cell	4 h
One streetlight out for a few days	Street Lighting (0.95)	Medium (50.0)	Electrical Maintenance Cell (PWD)	72 h

These are the built-in samples of scripts/demo_cli.py. In the second sample, four similar complaints in the seeded corpus triggered the +12 repeat-report boost. Analysing one complaint end to end (models, lexicon, duplicate search and routing) took about 9 ms on the development machine.

CHAPTER 11

TESTING AND VALIDATION

11.1 Testing support in the project

- The pytest suite contains 34 automated tests, and all of them pass: 9 API, 7 lexicon, 4 pipeline, 7 preprocessing and 7 routing tests.
- API tests use FastAPI's TestClient with an in-memory SQLite database, so the real database file is never touched.
- Routing tests verify the department mapping, SLA deadlines, critical escalation, the repeat-report boost and the 100-point cap.
- Pipeline tests check that a complete ticket is produced, that empty text is rejected and that a repeated complaint is detected as a duplicate.
- The evaluation script reports accuracy, precision, recall, F1, confusion matrices and ordinal MAE for the held-out test set.

11.2 Functional validation areas

Area	Expected behavior
Preprocessing	Text is lowercased, URLs and punctuation are removed, stop-words dropped and tokens stemmed.
Lexicon scoring	Emergency text scores above 0.8 and minor text below 0.4; repeats and durations are detected.
Classification	A complaint receives a category with confidence and an urgency class.
Duplicate detection	A resubmitted complaint is matched to its earlier copy.
Routing	Each category maps to its department; Critical tickets are escalated with a 4-hour SLA.
REST API	Valid complaints return 201, short text or an invalid status returns 422, unknown IDs return 404.
Dashboard statistics	Counts reflect the complaints that have been submitted.

11.3 Validation limitation

The frontend has no automated tests, and the system was validated only on synthetic data. The blend weights, score thresholds and department table are design choices that were not compared with real municipal outcomes.

CHAPTER 12

SECURITY AND PRIVACY CONSIDERATIONS

Because the application handles citizen-submitted text and location details, security and privacy are important aspects of the implementation.

12.1 Existing measures

- NLP processing is fully local; no external AI API or cloud service receives complaint text.
- The pipeline runs offline after installation because no NLTK corpora need to be downloaded.
- Pydantic schemas validate input: complaint text must be 10 to 4,000 characters and the location at most 200 characters.
- Status updates are restricted to Open, In Progress and Resolved.
- SQLAlchemy builds parameterized queries for all filters, which prevents SQL injection.
- CORS is limited to the local frontend origins (localhost:5173 and 127.0.0.1:5173).
- A lock protects the shared duplicate index against concurrent requests.

12.2 Deployment considerations

- The API has no authentication or role separation, so the staff endpoints must be protected before a real deployment.
- The CORS origins, the API address used by the frontend and the database URL should move to environment configuration.
- Complaint text may contain personal information; access control, retention rules and encryption at rest should be added.
- SQLite should be replaced by PostgreSQL or MySQL for concurrent multi-user use.
- Priority scores are decision-support indicators, and a human should review Critical or sensitive tickets.

CHAPTER 13

LIMITATIONS AND FUTURE SCOPE

13.1 Current limitations

- The dataset is synthetic and template-based, so the reported accuracy is not evidence of performance on real citizen complaints.
- The pipeline is English-only; code-mixed and regional-language complaints are not supported.
- Urgency is treated as four-class classification rather than ordinal regression.
- The blend weights, thresholds and repeat boost are hand-set, and the blend slightly lowered urgency accuracy on the test set (93.83% against 94.50%).
- The duplicate index is seeded with the synthetic corpus and lives in memory, so complaints submitted later are not remembered by the index after a restart.
- Each complaint receives exactly one category, even when it describes more than one issue.
- The API has no authentication and the frontend has no automated tests.

13.2 Future scope

- Validate and retrain the models on real municipal complaint data, including short, messy and code-mixed text.
- Add support for regional languages such as Hindi and Marathi.
- Fine-tune a transformer such as DistilBERT or IndicBERT while keeping the lexicon explanation layer.
- Model urgency with ordinal regression or an ordinal loss.
- Tune the blend weights and thresholds using staff feedback.
- Use active learning so that staff corrections to category and urgency feed periodic retraining.
- Store the duplicate index in the database and merge similar reports into a single ticket.
- Add staff authentication, role-based access, map-based views and frontend tests.

CHAPTER 14

CONCLUSION

CivicLens AI demonstrates a practical application of Natural Language Processing and machine learning to municipal complaint handling. The system integrates a FastAPI service, SQLite persistence, a React interface, supervised category and urgency classification, lexicon-based scoring, duplicate detection and a deterministic routing engine that turns raw complaint text into a prioritized, routed and explainable ticket.

The project uses different techniques for different sub-problems. TF-IDF with a calibrated Linear SVM identifies the category, TF-IDF combined with lexicon features and Logistic Regression estimates urgency, cosine similarity finds repeat reports, and a transparent scoring formula turns these signals into a priority score, a department and an SLA deadline.

On the held-out test set of 600 synthetic complaints, the category classifier reached 98.33% accuracy and the urgency classifier 94.50% accuracy with an ordinal MAE of 0.055, and both results were reproduced independently. Because the dataset is synthetic and template-based, these figures demonstrate the pipeline rather than real-world performance, and validation on real complaints is the most important next step.

Overall, the project provides a clear demonstration of how NLP and machine learning can be integrated into a useful civic application in which every priority decision can be traced back to its evidence.

CHAPTER 15

REFERENCES

[1] G. Salton, C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, 1988.

[2] M. F. Porter, "An algorithm for suffix stripping," Program, 1980.

[3] A. McCallum, K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification," AAAI-98 Workshop on Learning for Text Categorization, 1998.

[4] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," European Conference on Machine Learning (ECML), 1998.

[5] F. Sebastiani, "Machine Learning in Automated Text Categorization," ACM Computing Surveys, 2002.

[6] C. J. Hutto, E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," Proceedings of ICWSM, 2014.

[7] C. D. Manning, P. Raghavan, H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[8] P. A. Gutiérrez, M. Pérez-Ortiz, J. Sánchez-Monedero, F. Fernández-Navarro, C. Hervás-Martínez, "Ordinal Regression Methods: Survey and Experimental Study," IEEE Transactions on Knowledge and Data Engineering, 2016.

[9] L. Breiman, "Random Forests," Machine Learning, 2001.

[10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, 2011.

[11] J. Platt, "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods," Advances in Large Margin Classifiers, MIT Press, 1999.

[12] CivicLens AI source project, including the NLP pipeline, routing engine, FastAPI service, React frontend, trained models, automated tests and evaluation report.

[13] Synthetic complaint dataset generated by civiclens/data/generate_dataset.py (3,000 complaints, random seed 42).

— END OF REPORT —